



Université Hassan II de Casablanca
Faculté des Sciences Ben M'Sick (FSBM)

GUIDE D'INSTALLATION

Système de Détection d'Intrusion Temps Réel (GPU + Streaming)

Cyber-Sentinel

Étudiants : Bouazza Amina, El Betti Jihad, Hafsa El Hilali

Année universitaire : 2025 – 2026

19 juin 2026

Table des matières

Introduction du Guide	2
Étape 1 : Installation des Logiciels de Base	3
1 Vérification de Python 3.11	3
2 Installation de Npcap	3
2.1 Qu'est-ce que Npcap ?	3
2.2 Procédure d'installation	3
2.3 Vérification de l'installation	4
Étape 2 : Installation et Lancement de Kafka (Zookeeper + Broker)	5
1 Pourquoi Apache Kafka ?	5
2 Installation de Kafka	5
3 Lancement des Services	6
3.1 Terminal 1 — Lancement de Zookeeper	6
3.2 Terminal 2 — Lancement du Broker Kafka	6
Étape 3 : Organisation et Arborescence du Projet	7
1 Module 1 — Capture des flux réseau (<code>capture.py</code>)	7
2 Module 2 — Streaming avec Kafka (<code>producer.py</code>)	7
2.1 Résultat obtenu	7
3 Module 3 — Détection d'Intrusions par GPU (<code>detection.py</code>)	7
3.1 Résultat obtenu — La Preuve du GPU	8
3.2 Note : Compatibilité matérielle GPU vs CPU	8
4 Module 4 — Visualisation : Dashboard Cyber-Sentinel	8
4.1 Analyse des composants de l'interface	8
Conclusion du Guide	15

Introduction du Guide

Dans un paysage numérique où les cyberattaques deviennent de plus en plus sophistiquées et volumineuses, les systèmes de détection d'intrusion (IDS) traditionnels, basés uniquement sur le processeur (CPU), atteignent rapidement leurs limites. Face à des flux réseau de plusieurs gigabits par seconde, l'analyse séquentielle devient un goulot d'étranglement, augmentant la latence et laissant passer des menaces critiques.

Ce guide a pour objectif d'accompagner l'étudiant dans la conception et l'implémentation d'un système IDS de nouvelle génération, reposant sur trois piliers technologiques majeurs :

- **Le Streaming Distribué (Apache Kafka)** : Pour assurer le transport massif et sans perte des paquets réseau entre les différents nœuds du système.
- **Le Calcul Haute Performance (GPU / CUDA)** : Pour déporter l'analyse statistique et comportementale sur la carte graphique, permettant un traitement parallèle de milliers de flux simultanément via la bibliothèque PyTorch.
- **L'Observabilité en Temps Réel (Dash & Plotly)** : Pour offrir une interface de supervision moderne permettant une prise de décision rapide.

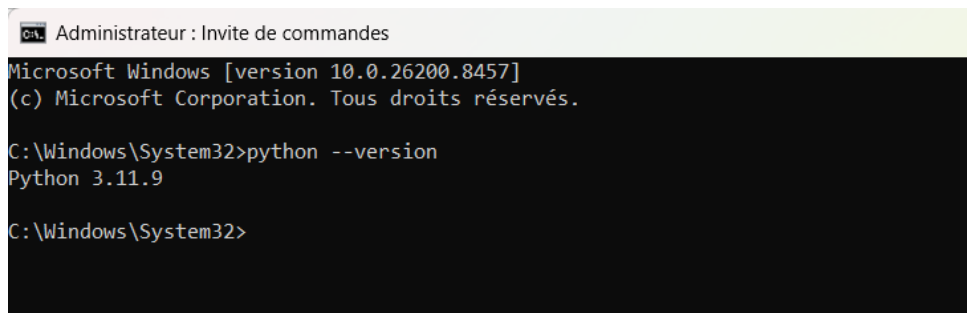
À travers les étapes de ce document, nous transformerons une simple capture réseau brute en une plateforme intelligente baptisée « **Cyber-Sentinel** », capable de classifier et de visualiser les menaces (DDoS, Port Scan, Flood) au moment précis où elles se produisent.

Étape 1 : Installation des Logiciels de Base

1 Vérification de Python 3.11

Ouvrez un terminal (CMD ou PowerShell) et tapez :

```
python -version ou py -3.11 -version
```



```
Administrator : Invite de commandes
Microsoft Windows [version 10.0.26200.8457]
(c) Microsoft Corporation. Tous droits réservés.

C:\Windows\System32>python --version
Python 3.11.9

C:\Windows\System32>
```

FIGURE 1 – Vérification de la version Python 3.11

On utilise la version 3.11 car c’est la version la plus stable pour PyTorch (GPU) et Kafka. Si vous avez la version 3.14, assurez-vous d’avoir également la version 3.11 installée en parallèle.

2 Installation de Npcap

2.1 Qu’est-ce que Npcap ?

Npcap est une bibliothèque logicielle (driver) qui permet de capturer et d’analyser le trafic réseau sur Windows. Dans notre projet, il sert de « **pont** » : il intercepte les paquets qui arrivent sur votre carte Wi-Fi ou Ethernet et les transmet à notre script Python (Scapy). Sans Npcap, Python est « aveugle » et ne peut pas voir les paquets circuler sur le réseau.

2.2 Procédure d’installation

Pour que le projet fonctionne, suivez rigoureusement ces étapes :

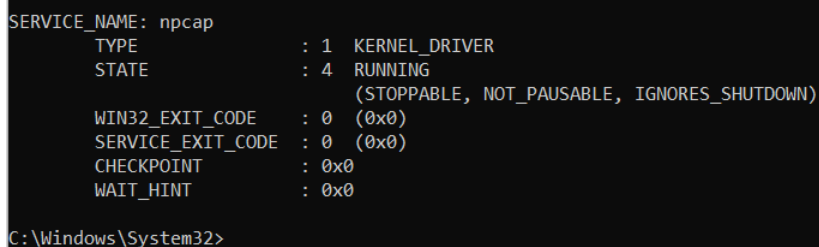
1. **Téléchargement** : Allez sur le site officiel npcap.com et téléchargez la dernière version du « Npcap Installer ».
2. **Lancement** : Exécutez le fichier `.exe` en tant qu’administrateur.
3. **Configuration CRITIQUE (Ne pas rater)** : Lors de l’installation, une fenêtre d’options s’affiche. Vous devez impérativement cocher la case suivante :
 - « **Install Npcap in WinPcap API-compatible Mode** »

— *Pourquoi ?* La bibliothèque Scapy utilise d’anciennes fonctions standards définies par l’API WinPcap.

2.3 Vérification de l’installation

Une fois l’installation terminée, il est inutile de chercher un logiciel à ouvrir. Npcap travaille en arrière-plan. Pour vérifier qu’il est bien actif, ouvrez un terminal (CMD) et tapez la commande suivante :

```
1 sc query npcap
```



```
SERVICE_NAME: npcap
        TYPE               : 1  KERNEL_DRIVER
        STATE                : 4  RUNNING
                             (STOPPABLE, NOT_PAUSABLE, IGNORES_SHUTDOWN)
        WIN32_EXIT_CODE       : 0  (0x0)
        SERVICE_EXIT_CODE   : 0  (0x0)
        CHECKPOINT           : 0x0
        WAIT_HINT            : 0x0

C:\Windows\System32>
```

FIGURE 2 – Résultat attendu de la commande `sc query npcap` — état RUNNING

Si l’installation a réussi, vous verrez s’afficher `STATE : 4 RUNNING`. Cela signifie que le pilote est chargé dans le noyau de Windows et prêt à capturer des données.

Conseil : Si vous recevez l’erreur `RuntimeError: Sniffing requires Npcap` lors du lancement du projet, cela signifie que cette étape a été sautée ou que la case « WinPcap compatibility » n’a pas été cochée. Dans ce cas, désinstallez Npcap et recommencez proprement.

Étape 2 : Installation et Lancement de Kafka (Zookeeper + Broker)

1 Pourquoi Apache Kafka ?

Kafka est un « Broker de messages » (un transporteur). Dans un système IDS professionnel, on ne peut pas envoyer les données directement au GPU car, en cas de trop grand nombre de paquets, le système va planter. Kafka sert de **tampon (Buffer)** : il stocke les paquets en mémoire et les distribue aux nœuds de détection à la vitesse qu'ils peuvent supporter.

2 Installation de Kafka

1. **Téléchargement** : Allez sur `kafka.apache.org` et téléchargez la version **Binary** (pas les sources).
2. **Extraction** : Dézippez le dossier directement à la racine de votre disque dur (ex : `C:\kafka`).
3. **Vérification Java** : Kafka nécessite Java. Vérifiez votre installation avec `java --version`.

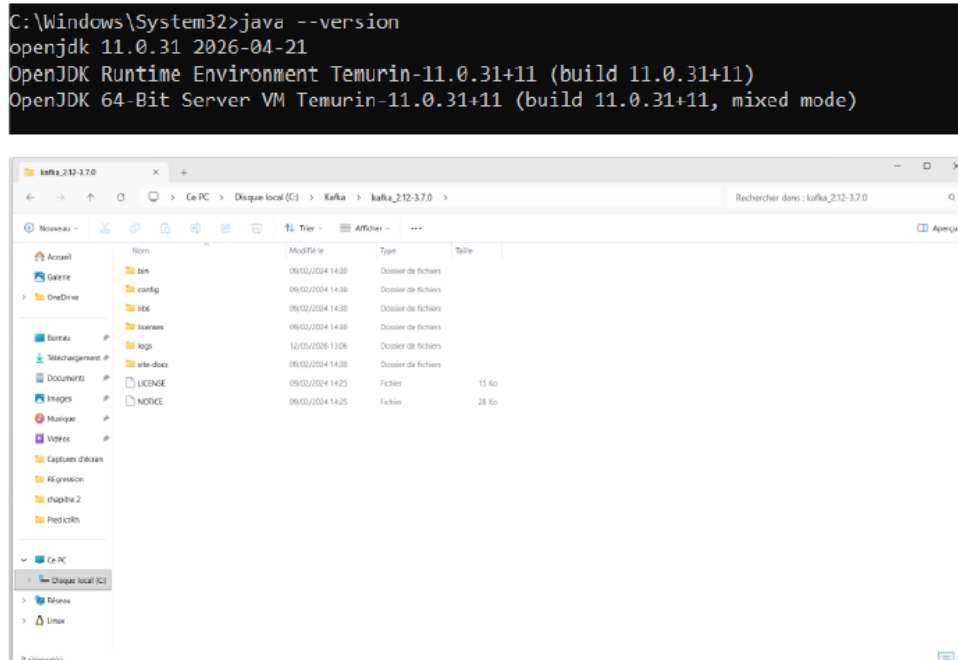


FIGURE 3 – Structure du dossier d'installation Kafka (`C:\kafka_2.12-3.7.0`)

Conseil : Kafka nécessite un environnement d'exécution Java (JRE/JDK). Vérifiez votre version avec `java --version`. Une version **11** ou **17** est recommandée pour garantir la stabilité du système réparti.

3 Lancement des Services

3.1 Terminal 1 — Lancement de Zookeeper

Zookeeper est le « chef d’orchestre » qui gère la coordination de Kafka. Ouvrez un premier terminal dans le répertoire `C:\kafka` et exécutez :

```
1 .\bin\windows\zookeeper-server-start.bat .\config\zookeeper.properties
```

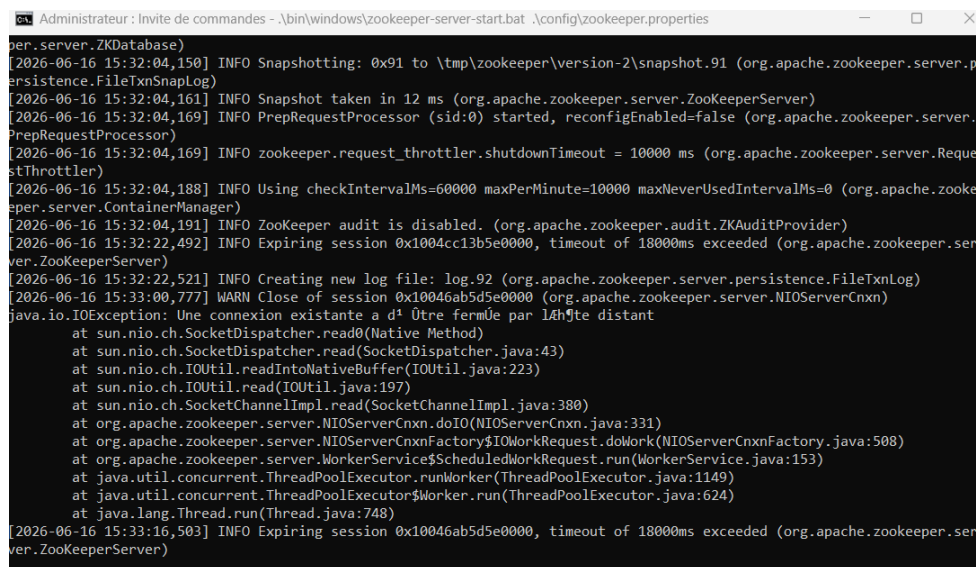


FIGURE 4 – Terminal Zookeeper : le service est actif et en écoute sur le port 2181

Vérification : Le terminal doit rester ouvert et afficher :
binding to port 0.0.0.0/0.0.0.0:2181

3.2 Terminal 2 — Lancement du Broker Kafka

C’est lui qui va réellement recevoir et envoyer nos paquets réseau. Ouvrez un **second terminal** et exécutez :

```
1 .\bin\windows\kafka-server-start.bat .\config\server.properties
```

Les deux terminaux doivent rester ouverts en permanence pendant toute la durée d’utilisation du projet.

Étape 3 : Organisation et Arborescence du Projet

Avant de commencer le développement, il est essentiel de structurer le projet de manière modulaire. Comme le montre la capture d'écran ci-dessous, le projet **IDS_PROJET** est divisé en **4 modules indépendants**, correspondant au pipeline de données :

Capture → Streaming → Analyse → Visualisation

1 Module 1 — Capture des flux réseau (capture.py)

Objectif : « Écouter » le réseau pour extraire les informations de chaque paquet (IP, Port, Taille) et les enregistrer.

Le script utilise la bibliothèque **Scapy** pour intercepter les paquets circulant sur l'interface réseau. Pour chaque paquet IP détecté, nous extrayons les métadonnées essentielles. L'enregistrement en format CSV permet de valider la structure des données avant de les envoyer dans le système distribué.

2 Module 2 — Streaming avec Kafka (producer.py)

Objectif : Transformer la capture locale en un flux de données distribué. Le but est de **séparer la capture de l'analyse** (découplage des services).

2.1 Résultat obtenu

Une fois le Producer lancé, le terminal affiche en temps réel les paquets interceptés par la sonde réseau. Cette capture confirme le bon fonctionnement de la couche de transport du système.

- **Capture en direct :** On observe que la machine locale (192.168.11.103) communique avec des serveurs distants (ex : 142.251.152.119).
- **Métadonnées extraites :** Pour chaque ligne, le script affiche le protocole (ici UDP) et la taille du paquet (ex : 1292 bytes). Ces informations sont cruciales pour détecter des anomalies de volume.
- **Publication Kafka :** Le préfixe [ENVOYÉ] indique que la donnée a été correctement sérialisée en JSON et publiée dans le topic Kafka `paquets_reseau`.

3 Module 3 — Détection d'Intrusions par GPU (detection.py)

Objectif : Créer le « **cerveau** » du système. Ce script consomme les données envoyées par le Producer (Module 2), les transforme en structures mathématiques (Tenseurs) et utilise la puissance de calcul du GPU pour détecter des comportements suspects.

Points clés du code :

- **Consommateur Kafka** : Le script s’abonne au topic `paquets_reseau`. Contrairement au Producer qui « pousse » les données, celui-ci les « tire » (Pull) pour les traiter.
- **PyTorch & CUDA** : On utilise `torch.device('cuda')` pour envoyer les statistiques de trafic sur la mémoire de la carte graphique.
- **Analyse par Tenseurs** : Au lieu de traiter les chiffres un par un, on les regroupe dans des matrices. Le GPU peut vérifier si 10 000 paquets respectent les seuils de sécurité en une fraction de seconde.

3.1 Résultat obtenu — La Preuve du GPU

- **Activation de l’accélération matérielle** : Le message `APPAREIL UTILISÉ : cuda` confirme que PyTorch a bien initialisé le moteur de calcul sur la carte graphique NVIDIA. Grâce aux cœurs CUDA, les calculs statistiques de chaque flux IP sont effectués en parallèle, garantissant une latence minimale.
- **État de préparation (Listening)** : Le message `Analyse des flux en cours...` indique que le consommateur Kafka est synchronisé avec le topic et attend l’arrivée des données.
- **Détection en temps réel** : On observe l’apparition immédiate d’alertes dès que les seuils sont franchis :
 - **ALERTE DDoS** : Détectée pour l’IP 192.168.11.103. Le GPU a calculé que le nombre de paquets par seconde dépassait la limite autorisée.
 - **ALERTE Flood** : Détectée pour l’IP 64.233.167.95. Le système a identifié un volume de données (octets) anormalement élevé, caractéristique d’une tentative de saturation de bande passante.

3.2 Note : Compatibilité matérielle GPU vs CPU

Bien que ce projet soit optimisé pour les cœurs CUDA des cartes graphiques NVIDIA, l’architecture a été conçue pour être **résiliente et portable**.

- **Mécanisme de repli (Fallback)** : Le code utilise `torch.device('cuda' if torch.cuda.is_available else 'cpu')`. Si aucun GPU compatible n’est détecté, le système bascule automatiquement sur le CPU sans erreur.
- **Performance vs Scalabilité** :
 - *Avec GPU* : Le système privilégie l’accélération verticale (un seul nœud très puissant).
 - *Sans GPU* : On peut exploiter la scalabilité horizontale de Kafka en lançant plusieurs instances du script `detection.py` sur différents ordinateurs pour répartir la charge.

Validation : Lors du lancement, si vous n’avez pas de GPU, vous verrez : `APPAREIL UTILISÉ : cpu`. Le Dashboard restera parfaitement fonctionnel.

4 Module 4 — Visualisation : Dashboard Cyber-Sentinel

Objectif : Passer d’un terminal de code complexe à une **interface de pilotage professionnelle**. Le Dashboard « Cyber-Sentinel » permet à un analyste de sécurité de comprendre l’état du réseau en un coup d’œil, sans avoir à lire des milliers de lignes de logs.

4.1 Analyse des composants de l’interface

1. Indicateurs Critiques (KPI Cards) :

- *Total Alerts (29)* : Un compteur en temps réel du nombre d’attaques détectées.

- *Primary Threat (DDoS)* : Le système identifie automatiquement l’attaque la plus fréquente.
- *Engine Mode (GPU CUDA)* : Une confirmation visuelle que le moteur de détection utilise l’accélération matérielle.

2. Journal des Intrusions (Event Logs) :

Ce tableau affiche l’historique précis des attaques. On y voit plusieurs tentatives de DDoS et de Flood provenant d’IP distinctes. La colonne *Details* montre la précision du calcul GPU (ex : « 1 ports / 20 paquets »), permettant de justifier chaque alerte déclenchée.

3. Répartition des Menaces (Threat Distribution) :

Le graphique en anneau (Donut Chart) offre une vue statistique. Dans cet exemple, le DDoS représente 55,2% des menaces, prouvant que le système sait classifier les types d’attaques dynamiquement.

4. Statut du Système (System Live) :

Le point vert clignotant et l’horloge confirment que le Dashboard est synchronisé avec le flux Kafka et affiche des données à la seconde près.

Architecture découplée : Cette interface n’est pas seulement un affichage esthétique ; c’est le résultat d’une architecture découplée. Le Dashboard peut être arrêté ou redémarré sans jamais interrompre la détection sur le GPU. C’est la force des Systèmes Répartis : chaque brique (Capture, Analyse, Visualisation) est indépendante et communique via le bus de messages Kafka.

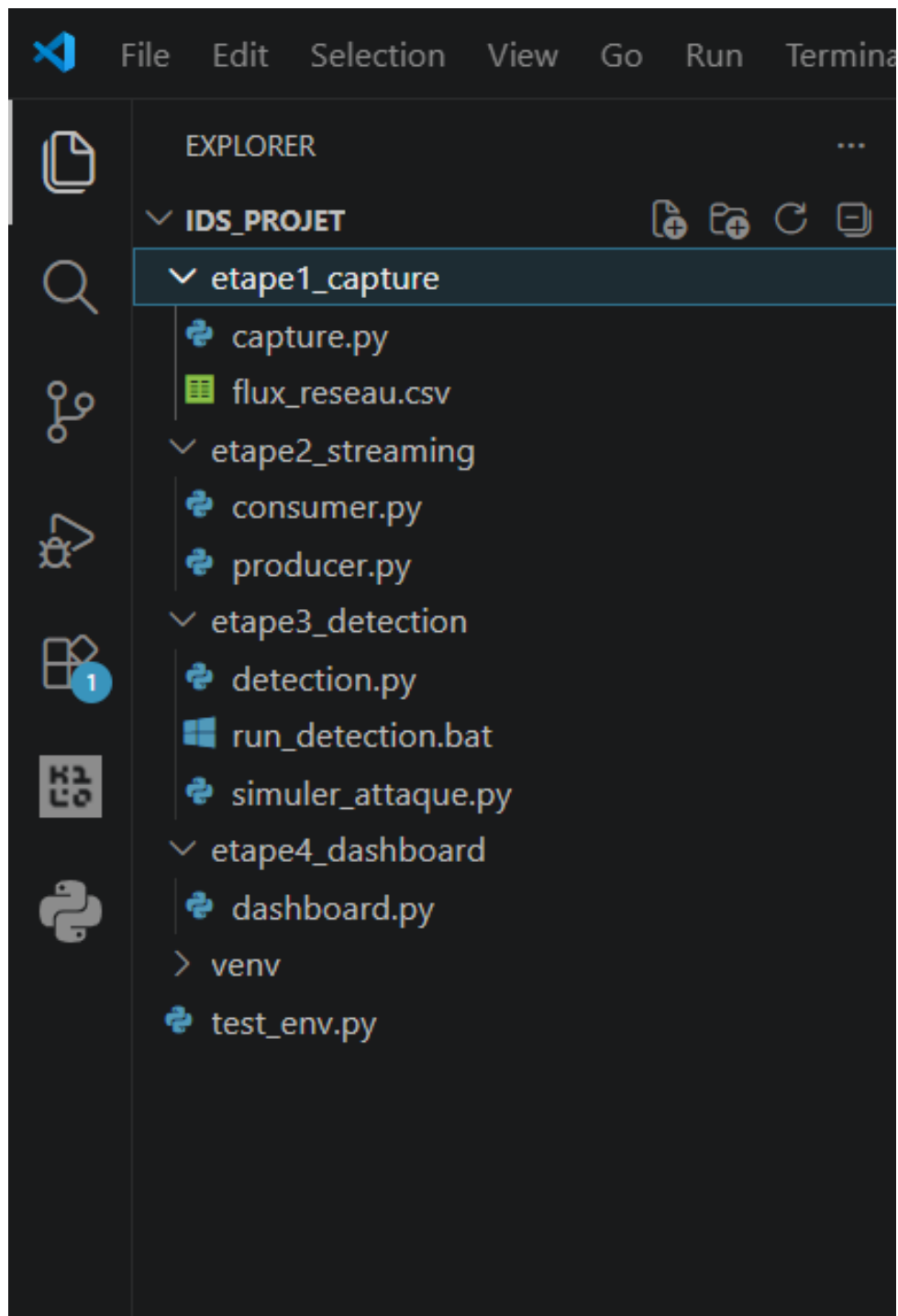
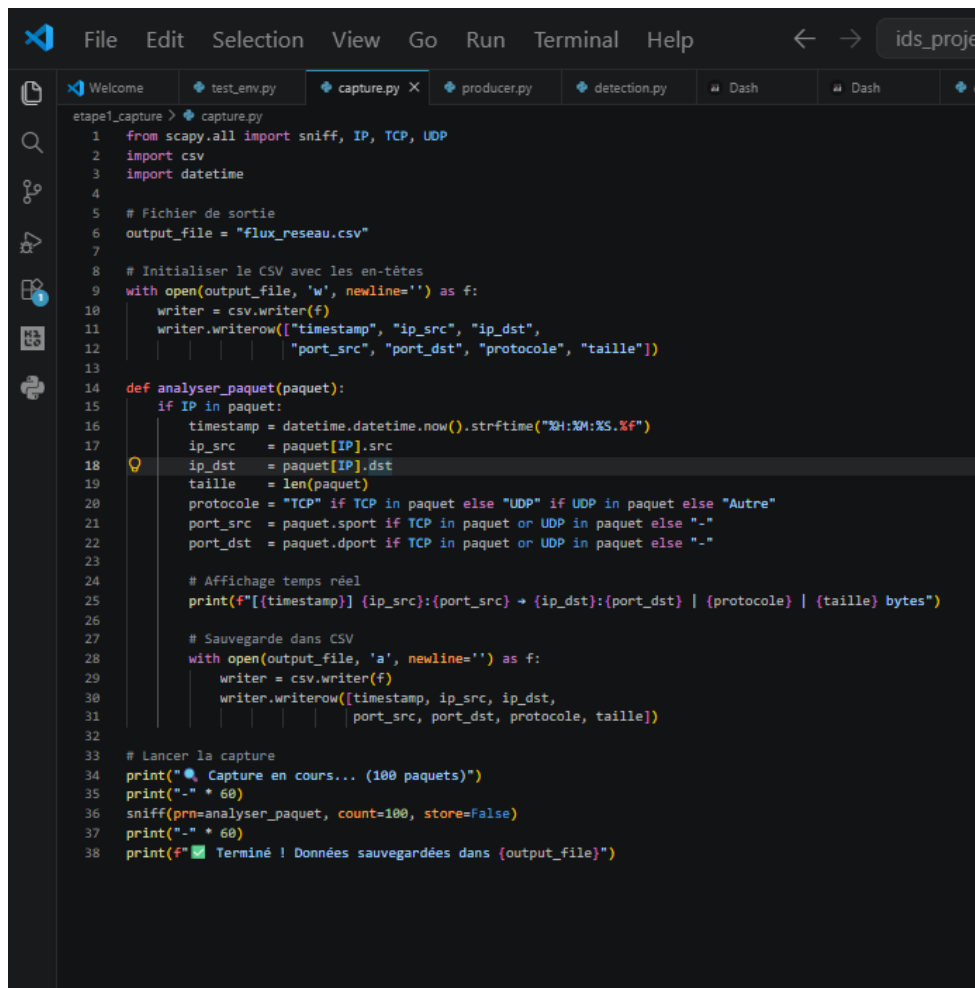


FIGURE 5 – Arborescence du projet IDS_PROJET : 4 modules indépendants



```
File Edit Selection View Go Run Terminal Help
Welcome test_env.py capture.py producer.py detection.py Dash Dash
etape1_capture > capture.py
1 from scapy.all import sniff, IP, TCP, UDP
2 import csv
3 import datetime
4
5 # Fichier de sortie
6 output_file = "flux_reseau.csv"
7
8 # Initialiser le CSV avec les en-têtes
9 with open(output_file, 'w', newline='') as f:
10     writer = csv.writer(f)
11     writer.writerow(["timestamp", "ip_src", "ip_dst",
12                     "port_src", "port_dst", "protocole", "taille"])
13
14 def analyser_paquet(paquet):
15     if IP in paquet:
16         timestamp = datetime.datetime.now().strftime("%H:%M:%S.%f")
17         ip_src = paquet[IP].src
18         ip_dst = paquet[IP].dst
19         taille = len(paquet)
20         protocole = "TCP" if TCP in paquet else "UDP" if UDP in paquet else "Autre"
21         port_src = paquet.sport if TCP in paquet or UDP in paquet else "-"
22         port_dst = paquet.dport if TCP in paquet or UDP in paquet else "-"
23
24         # Affichage temps réel
25         print(f"[{timestamp}] {ip_src}:{port_src} → {ip_dst}:{port_dst} | {protocole} | {taille} bytes")
26
27         # Sauvegarde dans CSV
28         with open(output_file, 'a', newline='') as f:
29             writer = csv.writer(f)
30             writer.writerow([timestamp, ip_src, ip_dst,
31                             port_src, port_dst, protocole, taille])
32
33 # Lancer la capture
34 print("🔍 Capture en cours... (100 paquets)")
35 print("-" * 60)
36 sniff(prn=analyser_paquet, count=100, store=False)
37 print("-" * 60)
38 print(f"✅ Terminé ! Données sauvegardées dans {output_file}")
```

FIGURE 6 – Code `capture.py` : utilisation de Scapy pour intercepter les paquets réseau

```

1 from scapy.all import sniff, IP, TCP, UDP
2 from kafka import KafkaProducer
3 import json
4 import datetime
5
6 producer = KafkaProducer(
7     bootstrap_servers='127.0.0.1:9092',
8     value_serializer=lambda x: json.dumps(x).encode('utf-8'),
9     # On ajoute un serializer pour la clé (string -> bytes)
10    key_serializer=lambda x: x.encode('utf-8'),
11    api_version=(3, 7, 0)
12)
13
14 def analyser_paquet(paquet):
15     if IP in paquet:
16         ip_source = paquet[IP].src # On stocke l'IP source
17         data = {
18             "timestamp" : datetime.datetime.now().strftime("%H:%M:%S.%f"),
19             "ip_src" : ip_source,
20             "ip_dst" : paquet[IP].dst,
21             "port_src" : paquet.sport if (TCP in paquet or UDP in paquet) else 0,
22             "port_dst" : paquet.dport if (TCP in paquet or UDP in paquet) else 0,
23             "protocole" : "TCP" if TCP in paquet else "UDP" if UDP in paquet else "Autre",
24             "taille" : len(paquet)
25         }
26
27         # IMPORTANT : On ajoute key=ip_source
28         # Cela garantit que la même IP va toujours vers le même Consumer
29         producer.send('paquets_reseau', key=ip_source, value=data)
30
31         print(f"[ENVOYÉ] {data['ip_src']} → {data['ip_dst']} | {data['protocole']} | {data['taille']} bytes")
32
33 print("🔥 Producteur démarré (Partitionnement par IP actif)...")
34 print("-" * 60)
35 sniff(prn=analyser_paquet, count=100, store=False) # Augmenté à 100 pour mieux voir les attaques
36 producer.flush()
37 print("✅ Paquets envoyés !")

```

FIGURE 7 – Code `producer.py` : publication des paquets dans Kafka avec partitionnement par IP

```

PS C:\WEITA\s2\systeme reparties\projet\ids_projet\etape2_streaming> py -3.11 producer.py
🔥 Producteur démarré (Partitionnement par IP actif)...
-----
[ENVOYÉ] 192.168.11.103 → 142.251.152.119 | UDP | 71 bytes
[ENVOYÉ] 192.168.11.103 → 64.233.167.95 | UDP | 1292 bytes
[ENVOYÉ] 192.168.11.103 → 64.233.167.95 | UDP | 1292 bytes
[ENVOYÉ] 192.168.11.103 → 64.233.167.95 | UDP | 119 bytes
[ENVOYÉ] 192.168.11.103 → 142.251.152.119 | UDP | 1231 bytes
[ENVOYÉ] 142.251.152.119 → 192.168.11.103 | UDP | 68 bytes
[ENVOYÉ] 142.251.152.119 → 192.168.11.103 | UDP | 75 bytes
[ENVOYÉ] 192.168.11.103 → 142.251.152.119 | UDP | 75 bytes

```

FIGURE 8 – Terminal `producer.py` : chaque paquet intercepté est publié dans Kafka ([ENVOYÉ])

```

52 try:
53     while True:
54         msg = consumer.poll(0.1)
55         if msg is None: continue
56         if msg.error(): continue
57
58         # Charger les données du paquet
59         paquet = json.loads(msg.value().decode('utf-8'))
60         ip_s = paquet['ip_src']
61         port_d = paquet['port_dst']
62         taille = paquet['taille']
63
64         # 1. Mise à jour des statistiques
65         stats[ip_s]['nb'] += 1
66         stats[ip_s]['ports'].add(port_d)
67         stats[ip_s]['vol'] += taille
68
69         # 2. CRÉATION DU TENSEUR PYTORCH (Analyse GPU)
70         features = torch.tensor([
71             float(stats[ip_s]['nb']),
72             float(len(stats[ip_s]['ports'])),
73             float(stats[ip_s]['vol'])
74         ]).to(device)
75
76         # 3. LOGIQUE DE DÉTECTION
77         alerte_detectee = False
78         type_attaque = ""
79
80         if features[0] >= SEUIL_DDOS:
81             type_attaque = "DDoS"
82             alerte_detectee = True
83         elif features[1] >= SEUIL_SCAN:
84             type_attaque = "Port Scan"
85             alerte_detectee = True
86         elif features[2] >= SEUIL_FLOOD:
87             type_attaque = "Flood"
88             alerte_detectee = True
89
90         # 4. ACTION
91         if alerte_detectee:
92             print(f" 🚨 ALERTE {type_attaque} détectée pour IP: {ip_s}")
93
94             # APPEL DE L'ENVOI AU DASHBOARD
95             envoyer_alerte_dashboard(
96                 type_attaque,
97                 ip_s,
98                 int(features[0]),
99                 int(features[1])
100             )
101
102             # Réinitialisation des stats pour éviter de spammer l'alerte
103             stats[ip_s] = {'nb': 0, 'ports': set(), 'vol': 0}
104         else:
105             # Affichage silencieux en mode console
106             print(f"[OK] {ip_s} -> {port_d} ({taille} bytes)", end='\r')
107
108     except KeyboardInterrupt:
109         print("\nArrêt du système de détection.")
110     finally:
111         consumer.close()

```

FIGURE 9 – Code detection.py : logique de détection par tenseurs et seuils

```
py - etape3_detection + v [] [] ... | {} x
PS C:\MEIA\s2\systeme reparties\projct\ids_projet\etape3_detection> py -3.1
1 detection.py
❖ PS C:\MEIA\s2\systeme reparties\projct\ids_projet\etape3_detection> py -3.1
1 detection.py
✅ MOTEUR DE DÉTECTION IDS ACTIF
✅ APPAREIL UTILISÉ : cuda
🔍 Analyse des flux en cours... (Prêt pour le Dashboard)
-----
🚨 ALERTE DDoS détectée pour IP: 192.168.11.103
🚨 ALERTE DDoS détectée pour IP: 64.233.167.95
🚨 ALERTE Flood détectée pour IP: 64.233.167.95
🚨 ALERTE DDoS détectée pour IP: 192.168.11.103
```

FIGURE 10 – Terminal `detection.py` : GPU CUDA activé et alertes DDoS/Flood déclenchées en temps réel

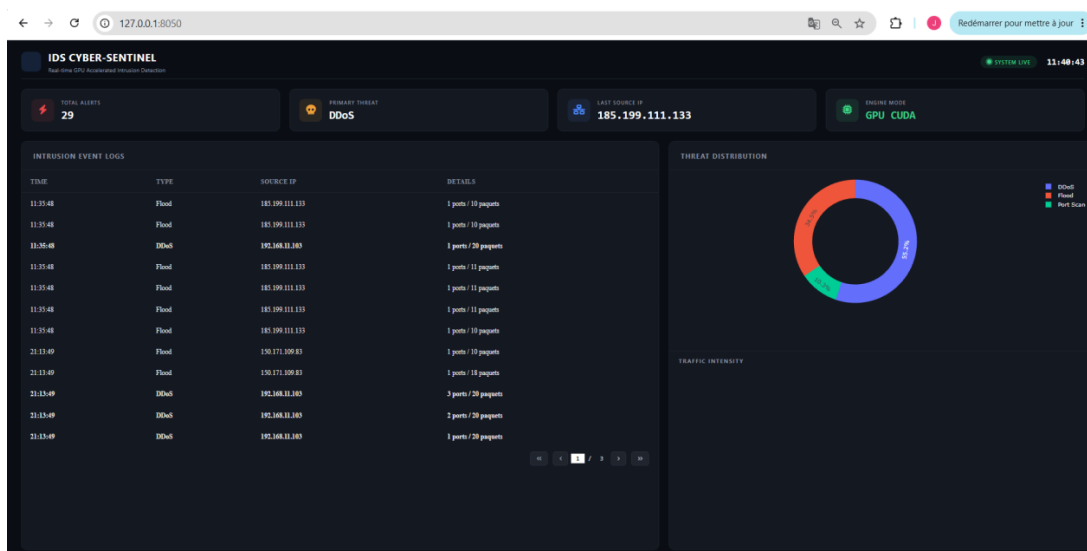


FIGURE 11 – Dashboard Cyber-Sentinel — Interface de supervision temps réel (127.0.0.1 :8050)

Conclusion du Guide

La réalisation de ce projet démontre qu'un système de détection d'intrusion performant ne repose pas seulement sur l'algorithme de détection, mais sur la robustesse de son **architecture distribuée**.

En parcourant ce guide, nous avons validé plusieurs concepts fondamentaux du module Systèmes Répartis :

- **Le découplage des services** : Grâce à Kafka, la sonde de capture, le moteur d'analyse et l'interface visuelle fonctionnent de manière indépendante, garantissant la résilience du système.
- **L'accélération matérielle** : L'utilisation de PyTorch CUDA prouve que le passage au calcul parallèle sur GPU est une réponse efficace à l'explosion du volume des données réseau (Big Data).
- **La distribution intelligente** : Le partitionnement des flux par clé IP a permis de maintenir la cohérence des données au sein d'un environnement multi-nœuds.

Le système final, bien que basé sur des seuils statistiques, pose les fondations nécessaires pour intégrer des modèles d'intelligence artificielle plus complexes (**Deep Learning**) sans modifier la structure du pipeline. En maîtrisant cet ensemble technologique — de Scapy à Dash en passant par Kafka et le GPU — l'étudiant dispose désormais d'une base solide pour concevoir des solutions de cybersécurité modernes, scalables et prêtes pour les défis de demain.